

LibrePlan JDK 11 → JDK 25 Migration Plan

Branch: `jdk11to25` Author: drafted with Claude Code, 2026-08-11

1. *Where we actually stand today*

This plan was written after auditing the current `jdk11to25` branch, not from memory of a previous conversation — the earlier plan was never saved. Findings:

Item	Current state
Runtime JDK used to build	11 (<code>.forgejo/workflows/ubuntu_24.04-jdk-11.yml</code> , <code>.github/workflows/ubuntu-24.04-jdk-11.yml</code>)
<code>maven-compiler-plugin</code> source/target	1.8/1.8 , hardcoded in root <code>pom.xml</code> (not even using <code><release></code>)
<code>maven-compiler-plugin</code> version	3.11.0
<code>maven-surefire-plugin</code> version	2.19.1 (2015, pre-JPMS)
Spring Framework	4.3.9.RELEASE (2017, EOL, <code>javax</code> namespace)
Spring Security	4.2.3.RELEASE
Hibernate Core	5.1.1.Final (2016)
Hibernate Validator	5.3.6.Final
Apache CXF	3.1.7 (2016)
ZK Framework	8.6.0.1
Servlet API	<code>javax.servlet-api</code> 3.1.0 (Servlet 3.1 / Tomcat 8 era)
Quartz	2.3.2 (fine, current-ish)
Guava	33.5.0-android (already modern)
<code>commons-io</code> / <code>commons-fileupload</code>	2.19.0 / 1.6.0 (already modern)
JasperReports	6.20.0 (already modern)
MySQL driver	5.1.46 (2018, ancient)
PostgreSQL driver	42.7.3 (current)
Jakarta namespace migration	not started — everything is still <code>javax.*</code>

There is already a **manual, tag-triggered Forgejo workflow**

(`.forgejo/workflows/ubuntu_24.04-jdk-25.yml`) that builds the whole reactor under Temurin 25 with the compiler still pinned to 1.8/1.8. It's unclear whether it has ever been run to completion — the only related pom change so far is bumping `commons-fileupload` to 1.6.0. Because source/target is still 8 and a source scan found no removed JDK APIs in application code (no `SecurityManager`, `sun.misc.Unsafe`, `javax.script/Nashorn`, `finalize()`, or CORBA usage), that workflow plausibly *compiles*. That is not the same as *runs correctly*: Spring 4's CGLIB proxying, Hibernate 5.1's bytecode enhancement/proxy generation, and ZK's reflection all rely on libraries that bundle old ASM versions and do illegal reflective access — things that fail at **runtime**, not compile time, once the JVM's default module encapsulation

tightens (JDK 16+) and keeps tightening through 17/21/25.

Implication for the plan: the risk here isn't really "can javac target 25" — it's whether decade-old Spring 4 / Hibernate 5.1 / CXF 3.1 / ZK 8 can run correctly on a JVM that's 14 years newer than they are. So this plan treats the JDK bump and the dependency uplift as one coupled effort, done in small, always-buildable steps, rather than two separate tracks.

2. Guiding rule for every phase

Every phase must end with `mvn clean install` green on its target JDK, and every phase must be its own commit (or small stack of commits) that leaves `main/jdk11to25` in a working, deployable state. No phase depends on a later phase's changes to compile. If a phase turns out to be too large to keep green, split it further rather than let it go red temporarily.

Within each phase below, steps are ordered so each one is independently buildable — you can stop after any step and still have a working reactor.

3. Phase overview

Phase 0	Baseline audit	[done — this document]
Phase 1	Build-tooling uplift (still JDK 11)	JDK 11 runtime
Phase 2	JDK 17 + javax-compatible dep uplift	JDK 17 runtime
Phase 3	JDK 21 currency pass	JDK 21 runtime
Phase 4	JDK 25 (target)	JDK 25 runtime
Phase 5	Jakarta EE namespace migration (stretch, post-25)	

Each numbered JDK phase is itself an LTS (11 → 17 → 21 → 25), so every intermediate state is something you could actually ship and run in production if the project needed to pause partway through.

Phase 0 — Baseline audit (this document)

- No code changes.
 - Deliverable: this plan, committed to the repo.
-

Phase 1 — Build-tooling uplift (stays on JDK 11)

Goal: make the build itself capable of targeting newer bytecode levels and running on newer JVMs, without changing the runtime JDK or any framework version yet. This is the lowest-risk phase and should be done first because several later phases are blocked by tooling that's simply too old to run under JDK 17+.

Steps (each independently buildable, still on JDK 11):

1. Bump `maven-compiler-plugin` 3.11.0 → latest 3.x (supports `--release` up through

very recent JDKs). Replace the hardcoded `<source>1.8</source>` `<target>1.8</target>` pair with a single `<maven.compiler.release>` property, initially set to 8 so behavior is unchanged. Verify: `mvn clean install`.

2. Bump `maven-surefire-plugin` 2.19.1 → latest 2.22.x/3.x line. Surefire 2.19 forks JVMs using internal APIs that break under JDK 17+'s stronger encapsulation; this must happen before any JDK bump or `mvn test` will fail to even launch on the newer JDK. Verify: full test suite still passes on JDK 11.
3. Bump `maven-war-plugin` 2.6 → latest 3.x. Low risk, packaging-only.
4. Confirm local/CI Maven itself is $\geq 3.9.x$ (Maven 3.8.7 is currently installed locally) — Maven's own launcher needs to run under the target JVM too. Update CI workflow `setup-java/Maven` cache steps if needed.
5. Run `mvn dependency:tree` and `mvn dependency:analyze` across all four modules and save the output as a reference baseline for phase 2 (used to spot which library is pulling in an old bundled ASM/CGLIB transitively).

Exit criteria: `mvn clean install` green on JDK 11, no behavior change, tooling now new enough to survive later JDK bumps.

Phase 2 — JDK 17 + javax-compatible dependency uplift

Goal: get a real, CI-gated green build **and** passing test suite running on JDK 17, using the last dependency generation that still speaks `javax.*` (so we deliberately avoid coupling the JDK bump to the Jakarta rename here — that's Phase 5).

Steps:

1. Set `maven.compiler.release` to 17. Fix any compile errors this alone surfaces (expected to be minimal/none given the source scan). Build on JDK 11 first (a JDK 11 `javac` can't target 17 — so this step itself requires switching the build JVM; do it together with step 2).
2. Switch the CI build JVM to Temurin 17 (re-enable and fix `.github/workflows/ubuntu-24.04-jdk-17.yml.disabled` and `.forgejo/workflows/ubuntu_24.04-jdk-17.yml`, make one of them the required PR gate).
3. Uplift Spring Framework 4.3.9 → **Spring Framework 5.3.x** (last `javax` line, actively patched into 2024, well-tested on JDK 17). Update `spring-orm`, `spring-web`, `spring-context-support`, `spring-test` together (they must stay in lockstep).
4. Uplift Spring Security 4.2.3 → **Spring Security 5.8.x** to match Spring 5.
5. Uplift Hibernate Core 5.1.1 → **Hibernate 5.6.x** (last 5.x line, `javax` namespace, JDK 17-safe bytecode enhancement). Uplift `hibernate-ehcache/hibernate-validator` in step with it.
6. Uplift Apache CXF 3.1.7 → **CXF 3.5.x** (`javax` namespace, JDK 17-tested).
7. Uplift `javax.servlet-api` 3.1.0 → **4.0.1** (Servlet 4.0 / Tomcat 9 baseline) and confirm the target servlet container (Tomcat version used in deployment) is at least Tomcat 9.

8. Re-run `mvn dependency:tree` diff against the Phase 1 baseline; confirm no library is still pulling in a pre-JDK9-safe ASM/CGLIB transitively. ZK 8.6.0.1 is the biggest unknown here — check ZK’s own JDK 17 compatibility notes/changelog before this step; a ZK point-release bump may be required even though it’s not strictly a “javax vs jakarta” concern.
9. Full regression pass: unit tests + a manual smoke test of the app (planner/Gantt views, workers, configuration — the modules already flagged as sensitive in earlier screenshots/testing) since Spring/ Hibernate/CXF proxy generation is exactly the kind of thing that compiles fine but misbehaves at runtime.

Exit criteria: JDK 17 CI workflow is green and required on PRs; JDK 11 workflow can be retired once the team is confident (keep it disabled-but- present for one release cycle as a safety net).

Phase 3 — JDK 21 currency pass

Goal: this hop should be much smaller than Phase 2 — JDK 17→21 has a far shorter list of behavior changes than 11→17, and no framework-generation change is required (Spring 5.3.x / Hibernate 5.6.x / CXF 3.5.x are all JDK 21-compatible).

Steps:

1. Set `maven.compiler.release` to 21.
2. Add `.forgejo/workflows/ubuntu_24.04-jdk-21.yml` and a matching GitHub workflow, modeled on the JDK 17 one from Phase 2.
3. Patch-bump any dependency that has a JDK 21-specific fix released since the Phase 2 versions were pinned (check Spring 5.3.x latest patch, Hibernate 5.6.x latest patch).
4. Run the same regression pass as Phase 2 step 9.

Exit criteria: JDK 21 CI green and required; JDK 17 workflow kept as a secondary/non-blocking check for one cycle, then retired.

Phase 4 — JDK 25 (target)

Goal: reach the actual target. Same shape as Phase 3 — small delta expected if Phase 2/3 were done properly, since the framework versions chosen in Phase 2 were already selected for their JDK 17+ track record.

Steps:

1. Set `maven.compiler.release` to 25.
2. Promote `.forgejo/workflows/ubuntu_24.04-jdk-25.yml` from its current tag-only/manual `workflow_dispatch` trigger to a real `pull_request`/ push gate (mirror the trigger block used by the JDK 11/17 workflows), now that it’s backed by dependency versions actually chosen for this JDK rather than an untested compiler-flag change on

ancient libraries.

3. Watch specifically for: further default-encapsulation tightening, deprecated-for-removal APIs flagged in 21→25 release notes, and any `--add-opens/--add-exports` flags that become newly necessary for CGLIB/Hibernate proxy generation (add them to `argLine` in Surefire and to the app's launch script if so — this is the single most likely class of failure given Spring/Hibernate's use of reflection).
4. Full regression pass (same as Phase 2 step 9) plus a real deployment smoke test in the Docker environment this project runs in.
5. Retire the JDK 21 workflow once JDK 25 has been the required gate for a full release cycle.

Exit criteria: JDK 25 is the CI-required build/runtime JDK; `jdk11to25` branch merges to `main`.

Phase 5 — Jakarta EE namespace migration (stretch, do after Phase 4)

Not required to reach JDK 25 — Spring 5.3.x / Hibernate 5.6.x / CXF 3.5.x / Tomcat 9 all run on JDK 25 while still speaking `javax.*`. But those are the **last** `javax`-generation releases; they're increasingly EOL and won't get further CVE patches indefinitely. Flagging as explicit follow-on work so it doesn't get silently forgotten once "JDK 25" is checked off:

- Spring 5 → Spring 6 (`javax.*` → `jakarta.*`)
- Hibernate 5 → Hibernate 6
- CXF 3 → CXF 4
- Servlet 4 → Servlet 5/6 (Tomcat 9 → Tomcat 10/11)
- Every `javax.*` import in application code (annotations, JPA, servlet, validation, JAXB) needs mechanical renaming — likely scriptable with OpenRewrite's Jakarta EE recipes rather than by hand.
- ZK framework's Jakarta-compatible release line needs confirming (may force a ZK major version bump).

This is materially larger than Phases 1–4 combined and touches application code directly (not just poms), so it should be scoped as its own follow-up plan once Phase 4 has shipped.

4. Tracking

Suggested commit/PR granularity: one PR per numbered step within a phase (not one PR per phase) so review stays small and `git bisect` stays useful if a step turns out to break something Surefire didn't catch.